

Documento de Arquitetura de Software (DAS)

Visão Geral

O Smart Decisions é um sistema com arquitetura orientada a microsserviços de responsabilidades independentes. Ele utiliza serviços inteligentes para análise de modelos de negócios baseado no Business Canvas Model (BMC).

O sistema é composto pelos seguintes serviços principais:

- API Gateway: ponto único de entrada, conecta-se ao Canvas Service
- Canvas Service: gerencia os dados estruturados do Canvas e valida regras de negócio.
- Intelligence Service: executa o pipeline de IA, incluindo RAG, construção de prompt e integração com LLM.
- Market Data Service: integra com APIs externas de análise de mercado e normaliza os dados.
- Notification Service: envia notificações ao usuário quando a análise é concluída ou quando ocorre erro.
- Auth Service: Valida tokens, evita duplicação de tokens e realiza autenticação com todos os microsserviços do sistema.

Justificativa da Stack Tecnológica:

- Node.js/FastAPI: Integração com APIs e microsserviços, eficiência em operações I/O
- Java: Forte suporte às regras de negócio e CRUD.
- Python: Integração com os LLMs e sistemas inteligentes
- OAuth2: Colabora com o hardening do sistema
- Redis: Armazenamento temporário de dados por 24h (Cache do sistema)
- Oracle Database: Suporte para banco de dados relacional e vetorial
- RabbitMQ/Kafka/SQS: Responsável pela mensageria assíncrona
- Jinja: Construção do prompt final para o LLM
- Docker/Kubernetes: Ideal para sistemas de escalabilidade horizontal

Atributos de Qualidade

Escalabilidade:

- Cada microsserviço pode ser escalado horizontalmente via Kubernetes.
- Serviços mais demandados (Intelligence e Market Data) podem ter réplicas independentes.

- Mensageria desacopla picos de requisições do tempo de processamento da IA.
- Cache em Redis reduz chamadas repetidas a APIs externas.

Segurança:

- Autenticação baseada em JWT validado pelo AuthService.
- Não há envio direto de dados sensíveis para APIs externas sem pré-processamento e anonimato.
- Logs de auditoria das chamadas à LLM para rastreabilidade.

Resiliência

- Processamento assíncrono evita bloqueio da interface do usuário.
- Circuit Breaker no Market Data Service para falhas em APIs externas.
- Múltiplas tentativas controladas no consumo de eventos.
- Eventos de falha podem ser publicados e notificados para monitoramento.

Decisões de Design

ADR-01 — Uso de Mensageria para Processamento de IA

Decisão: Utilizar RabbitMQ/Kafka/SQS para acionar o processamento de IA de forma assíncrona.

Justificativa:

- O tempo de resposta da LLM é imprevisível.
- Evita bloquear o usuário e melhora experiência.
- Permite reprocessamento e escalabilidade de consumidores.
- Paralelismo eficiente

ADR-02 — Uso de RAG com Base Vetorial

Decisão: Incorporar RAG para complementar o conhecimento da LLM.

Justificativa:

- Reduz alucinação.
- Permite uso de conhecimento específico do domínio.
- Facilita atualização sem treinar modelo novamente.

ADR-03 — Separação do Market Data Service

Decisão: Isolar integrações externas em um microserviço próprio.

Justificativa:

- APIs externas são instáveis e variáveis.
- Evita impacto direto no pipeline de IA.
- Facilita cache e controle de custos.

ADR-04 — Linguagens Diferentes por Serviço

Decisão: Usar Java, Node.js/FastAPI e Python conforme a função do serviço.

Justificativa:

- Python é mais adequado para IA e automação
- Node.js/FastAPI são mais eficientes para I/O e gateways.
- Java é robusto para domínio e persistência transacional + CRUD rígido.